

## Day\_2\_Gates\_Check

August 27, 2026

```
[7]: from qiskit import QuantumCircuit
      from qiskit.circuit.library import XGate, YGate, ZGate, SGate, TGate
      import numpy as np
```

```
[25]: def unitary_check(name, matrix):
        #np.eye(2) creates a 2 x 2 identity matrix
        identity = np.eye(matrix.shape[0])

        #This calculates the conjugate transpose of the matrix.
        result = np.allclose(matrix.conj().T @ matrix, identity)

        print(f"{name} Gate:")
        print(matrix)
        print("Unitary:", result)
        print("-" * 50)
```

```
[26]: # X Gate
        #Quantum equivalent of a NOT gate
        qc = QuantumCircuit(1)
        qc.x(0)
        unitary_check("X", XGate().to_matrix())
        qc.draw()
```

```
X Gate:
[[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
Unitary: True
-----
```

```
[26]:
q:   X
```

```
[27]: # Y Gate
        #Performs a 180° rotation around the Y-axis of the Bloch sphere, with a phase
        ↪ change
        qc = QuantumCircuit(1)
        qc.y(0)
```

```
unitary_check("Y", YGate().to_matrix())
qc.draw()
```

```
Y Gate:
[[ 0.+0.j -0.-1.j]
 [ 0.+1.j  0.+0.j]]
Unitary: True
-----
```

[27]:

```
q:  Y
```

```
[28]: # Z Gate
      #Performs a 180° rotation around the Z-axis
      qc = QuantumCircuit(1)
      qc.z(0)
      unitary_check("Z", ZGate().to_matrix())
      qc.draw()
```

```
Z Gate:
[[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
Unitary: True
-----
```

[28]:

```
q:  Z
```

```
[29]: # S Gate
      #A phase gate that rotates the state by 90° (/2) around the Z-axis
      qc = QuantumCircuit(1)
      qc.s(0)
      unitary_check("S", SGate().to_matrix())
      qc.draw()
```

```
S Gate:
[[1.+0.j 0.+0.j]
 [0.+0.j 0.+1.j]]
Unitary: True
-----
```

[29]:

```
q:  S
```

```
[22]: # T Gate
      # /4 phase gate (45° rotation around Z-axis)
```

```
qc = QuantumCircuit(1)
qc.t(0)
unitary_check("T", TGate().to_matrix())
qc.draw()
```

T Gate:

```
[[1.      +0.j      0.      +0.j      ]
 [0.      +0.j      0.70710678+0.70710678j]]
```

Unitary: True

-----

[22]:

```
q: T
```

```
[30]: # CZ Gate
      #A two-qubit controlled-Z gate
qc = QuantumCircuit(2)
qc.cz(0, 1)
unitary_check("CZ", CZGate().to_matrix())
qc.draw()
```

CZ Gate:

```
[[ 1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  1.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]]
```

Unitary: True

-----

[30]:

```
q_0:
```

```
q_1:
```

```
[31]: # Toffoli (CCX) Gate
      #A three-qubit controlled-controlled-X gate
qc = QuantumCircuit(3)
qc.ccx(0, 1, 2)
unitary_check("Toffoli (CCX)", CCXGate().to_matrix())
qc.draw()
```

Toffoli (CCX) Gate:

```
[[1.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [0.+0.j  1.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [0.+0.j  0.+0.j  1.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  0.+0.j  1.+0.j]
 [0.+0.j  0.+0.j  0.+0.j  0.+0.j  1.+0.j  0.+0.j  0.+0.j  0.+0.j]]
```

```
[0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 1.+0.j 0.+0.j 0.+0.j]
[0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 1.+0.j 0.+0.j]
[0.+0.j 0.+0.j 0.+0.j 1.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]]
Unitary: True
-----
```

[31]:

q\_0:

q\_1:

q\_2: X